

Topic-1: Introduction to Arrays in JAVA

- **Java array** is an object which contains elements of a similar data type.
- Additionally, the elements of an array are stored in a contiguous memory location.
- It is a data structure where we store similar elements.
- We can store only a fixed set of elements in a Java array
- Array in Java is index-based, the first element of the array is stored at the index-0, the second element is stored at index-1 and so on.

1. Declaration of Arrays

To declare an array in Java, you specify the type of the elements it will hold, followed by square brackets [].

Examples:

```
// Declaring an array of integers
int[] numbers;
```

```
// Declaring an array of strings
String[] names;
```

```
// Declaring an array of doubles
double[] prices;
```

2. Initialization of Arrays

Initialization involves creating the actual array and optionally assigning values to it. There are several ways to initialize an array:

a. Declaration and Initialization in One Step

You can declare and initialize an array in a single statement using curly braces {} to specify the initial values:

```
// Array of integers with initial values
int[] numbers = {1, 2, 3, 4, 5};
```

```
// Array of strings with initial values
String[] names = {"Bobby", "Charan", "Alekhya"};
```

```
// Array of doubles with initial values
double[] prices = {40.5, 27.75, 30.0};
```

b. Declaration Followed by Initialization

You can declare an array first and then initialize it separately. This is done using the `new` keyword and specifying the size of the array:

```
// Declaration
int[] numbers;
```

```
// Initialization with size
numbers = new int[5]; // Creates an array with 5 elements, all
initialized to 0

// You can also initialize elements individually
numbers[0] = 1;
numbers[1] = 2;
numbers[2] = 3;
numbers[3] = 4;
numbers[4] = 5;
```

Topic-2: Storage of Array in Computer Memory

1. Memory Layout of Arrays:

a. Array Reference

In Java, arrays are objects. When you create an array, you first allocate memory for an array object which holds the reference to the actual array data. This reference points to a memory location where the actual array data is stored.

- **Reference:** The reference variable of the array holds the address of the array object in memory.

Example:

```
int[] numbers = new int[5];
// 'numbers' holds the reference to the array object.
```

b. Array Object

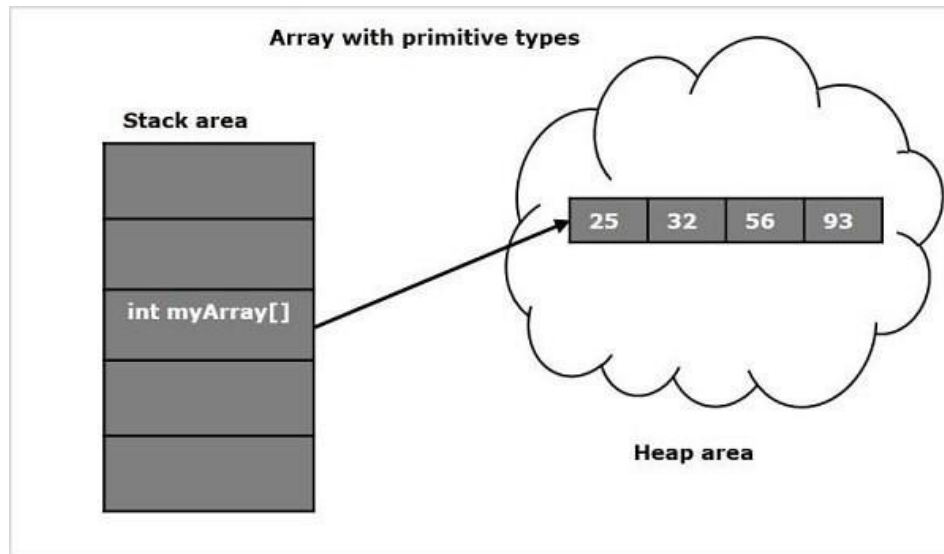
The array object itself contains metadata and the actual array data.

- **Metadata:** The array object contains metadata such as the length of the array and type information.
 - **Length:** The length of the array can be accessed using the `.length` property.
 - **Type:** Java maintains type information internally to ensure type safety.
- **Array Data:** The array object holds the actual data elements in contiguous memory locations. For instance, if you have an `int[]`, the array data will be stored in a contiguous block of memory, with each integer occupying 4 bytes.

The reference types in Java are stored in **heap area**. Since arrays are reference types (we can create them using the `new` keyword) these are also stored in heap area.

Example:

```
int MyArray[]=new int[4];  
MyArray={25,32,56,93};
```



2. Contiguous Memory Allocation

When an array is created, the Java runtime allocates a contiguous block of memory for the array elements. This means that if you have an array of 100 integers, there will be 100 consecutive memory locations reserved for storing those integers.

- **Example:** For `int[] numbers = new int[5];`, Java allocates a block of memory sufficient to store 5 integers (typically 20 bytes, since each `int` is 4 bytes).

3. Indexing and Access

Arrays in Java use zero-based indexing. The memory addresses of array elements can be computed based on the base address of the array and the size of each element. This allows for efficient access using an index.

Formula: The memory address of an element at index i can be calculated as:

$$\text{Address_of_element_i} = \text{Base_address} + (i * \text{size_of_element})$$

For instance, if the base address of the array is `0x1000` and each integer occupies 4 bytes, the address of the element at index 2 would be:

$$\text{Address_of_element_2} = 0x1000 + (2 * 4) = 0x1008$$

Topic-3: Accessing Elements of Arrays in Java

- Once the array is created, you can access an array element by using the name of the array followed by an index enclosed between a pair of square brackets.
- The index or subscript of an array indicates the position of an element in the array.
- The index of the first element in the array is always 0 (zero), the second element has an index 1 and so on.
- The index of the last element is always one less than the number of elements in the array.

Syntax:

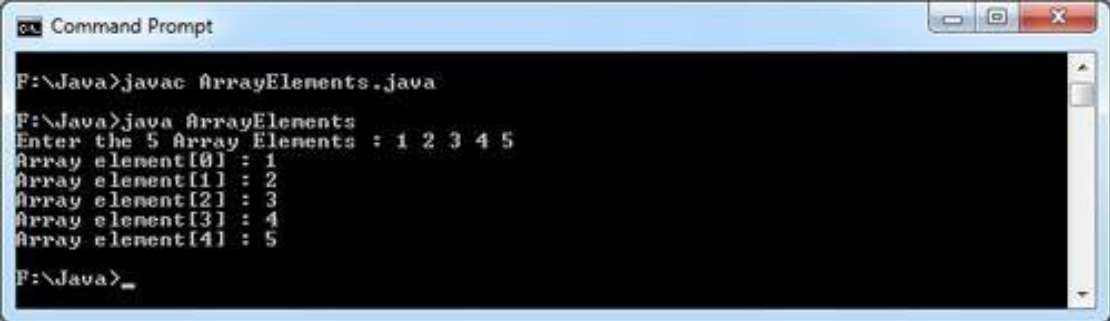
```
arrayRefVar[index];
```

Here, index is an integer literal or an expression that evaluates to an integer

Example:

```
import java.util.Scanner; //program uses Scanner class
public class ArrayElements {
    public static void main(String[] args) {
        int[] numArray = new int[5];
        int i;
        Scanner input=new Scanner(System.in);
        System.out.print("Enter the 5 Array Elements : ");
        for(i=0; i<5; i++)
            numArray[i]=input.nextInt(); //Read number
        for(i=0; i<numArray.length; i++)
            System.out.println("Array element[" + i + "] : "
+numArray[i]);
    }
}
```

Output:



```
Command Prompt
F:\Java>javac ArrayElements.java
F:\Java>java ArrayElements
Enter the 5 Array Elements : 1 2 3 4 5
Array element[0] : 1
Array element[1] : 2
Array element[2] : 3
Array element[3] : 4
Array element[4] : 5
F:\Java>_
```

3.1. Determination of Array Size:

To determine the size of a Java array, query its *length* property.

Here is an example of how to access the size of a Java array in code:

```
int[] exampleArray = {1,2,3,4,5};
int exampleArraySize = exampleArray.length;
System.out.print("This Java array size is: " + exampleArraySize );
//The Java array length example prints out the number 5
```

Note: The array 'length' in Java is a property, not a method. That means you can't follow the word length with round brackets.

3.2. Iterating through array elements using for-each loop

In Java, the **for-each** loop is used to iterate through elements of arrays

Example 1: Print Array Elements

```
// print array elements

class Main {
    public static void main(String[] args) {

        // create an array
        int[] numbers = {3, 9, 5, -5};

        // for each loop
        for (int number: numbers) {
            System.out.println(number);
        }
    }
}
```

Output

```
3
9
5
-5
```

Example 2: Sum of Array Elements

```
// Calculate the sum of all elements of an array

class Main {
    public static void main(String[] args) {

        // an array of numbers
        int[] numbers = {3, 4, 5, -5, 0, 12};
        int sum = 0;

        // iterating through each element of the array
        for (int number: numbers) {
            sum += number;
        }

        System.out.println("Sum = " + sum);
    }
}
```

Output:

```
Sum = 19
```

Topic-4: Passing an array as method parameter

Arrays can be passed to other methods just like how you pass primitive data type's arguments. To pass an array as an argument to a method, you just have to pass the name of the array without square brackets. The method prototype should match to accept the argument of the array type.

Given below is the method prototype:

```
void method_name (int [] array);
```

This means method_name will accept an array parameter of type int. So if you have an int array named myarray, then **you can call the above method as follows:**

```
method_name (myarray);
```

- The above call passes the reference to the array myarray to the method 'method_name'.
- Thus, **the changes made to myarray inside the method will reflect in the calling method as well.**

Example: Program to find maximum and minimum element of an array

```
import java.util.Scanner;
public class ArrayParameter {
    public int max(int [] arr) {
        int max = arr[0];

        for(int i=1; i<arr.length; i++ ) {
            if(arr[i]>max) {
                max = arr[i];
            }
        }
        return max;
    }

    public int min(int [] arr) {
        int min = arr[0];

        for(int i = 1; i<arr.length; i++ ) {
            if(arr[i]<min) {
                min = arr[i];
            }
        }
        return min;
    }

    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the array size:");
        int size = sc.nextInt();
        int[] arr = new int[size];
        System.out.println("Enter the elements of the array ::");

        for(int i=0; i<size; i++) {
            arr[i] = sc.nextInt();
        }
    }
}
```

```
        ArrayParameter m = new ArrayParameter();
        System.out.println("Maximum value in the array
is::"+m.max(arr));
        System.out.println("Minimum value in the array
is::"+m.min(arr));
    }
}
```

Output:

```
Enter the array size:
6
Enter the elements of the array ::
25
-8
3
34
-6
19
Maximum value in the array is::34
Minimum value in the array is::-8
```

Topic-5: Assigning one Array to another Array

Array in Java can be copied to another array using the following ways.

Scenario-1: Using variable assignment. This method has side effects as changes to the element of an array reflects on both the places. To prevent this side effect following are the better ways to copy the array elements.

Scenario-2: Create a new array of the same length and copy each element.

Scenario-3: Use the clone method of the array. Clone methods create a new array of the same size.

Scenario-4: Use System.arraycopy() method. The *arraycopy()* can be used to copy a subset of an array.

Example:

```
public class Tester {
    public static void main(String args[]) {
        //Scenario 1: Copy array using assignment
        int[] a = {1, 2, 3};
        int[] b = a;

        //test side effect
        b[0]++;

        System.out.println("Scenario 1: ");
        System.out.print("Array a: ");
        printArray(a);
        System.out.print("Array b: ");
        printArray(b);

        //Scenario 2: Copy array by iterating
        int[] c = {1, 2, 3};        int[] d = new int[c.length];
        for (int i = 0; i < d.length; i++) {
```

```
        d[i] = c[i];
    }

    //test side effect
    d[0]++;

    System.out.println("Scenario 2: ");
    System.out.print("Array c: ");
    printArray(c);
    System.out.print("Array d: ");
    printArray(d);

    //Scenario 3: Copy array using clone
    int[] e = {1, 2, 3};
    int[] f = e.clone();

    //test side effect
    f[0]++;

    System.out.println("Scenario 3: ");
    System.out.print("Array e: ");
    printArray(e);
    System.out.print("Array f: ");
    printArray(f);

    //Scenario 4: Copy array using arraycopy
    int[] g = {1, 2, 3};
    int[] h = new int[g.length];
    System.arraycopy(g, 0, h, 0, h.length);

    //test side effect
    h[0]++;

    System.out.println("Scenario 4: ");
    System.out.print("Array g: ");
    printArray(g);
    System.out.print("Array h: ");
    printArray(h);
}

public static void printArray(int[] a) {
    System.out.print("[ ");
    for (int i = 0; i < a.length; i++) {
        System.out.print(a[i] + " ");
    }
    System.out.println("]");
}
}
```

Output:

```
Scenario 1:
Array a: [ 2 2 3 ]
Array b: [ 2 2 3 ]

Scenario 2:
Array c: [ 1 2 3 ]
Array d: [ 2 2 3 ]

Scenario 3:
Array e: [ 1 2 3 ]
Array f: [ 2 2 3 ]
```

```
Scenario 4:  
Array g: [ 1 2 3 ]  
Array h: [ 2 2 3 ]
```

Topic-6: Dynamic change of array size in java

The size of an array is the number of elements it can hold. In Java, the array size is fixed when it is created.

The easiest way to resize an array is to create a new array with the desired size and copy the contents of the old array into it. We can delete the old array by setting it to null.

For this, we use the `arraycopy()` method of the `System` class or `copyOf()` method of the `java.util.Arrays` class.

Syntax of arraycopy():

```
void arraycopy(Object src, int srcPos, Object dest, int destPos, int length)
```

Parameters:

- `src`: This is the source array.
- `srcPos`: This is the starting position in the source array.
- `dest`: This is the destination array.
- `destPos`: This is the starting position in the destination array.
- `length`: This is the number of elements to be copied.

Example:

```
import java.util.Arrays;  
  
class HelloWorld {  
    public static void main( String args[] ) {  
        int[] oldArray = {1, 2, 3};  
        int[] newArray = new int[6];  
        System.arraycopy(oldArray, 0, newArray, 0, 3);  
        oldArray = null;  
        System.out.println(Arrays.toString(newArray));  
    }  
}
```

Output:

```
[1, 2, 3, 0, 0, 0]
```

Topic-7: Sorting of Arrays

1. Bubble Sort:

Bubble Sort is the elementary sorting algorithm for sorting various data structures. It is a comparison-based sorting algorithm in which each element is compared with the next element, and is swapped if those elements are not in the correct order.

Time Complexity : $O(N^2)$

Space Complexity : $O(1)$

Algorithm for Bubble Sort in Java

Step1: Repeat step 1 to 4 for $i=0$ to n

Step2: For $j=0$ to $n-i-1$

Step3: if($arr[j] > arr[j+1]$)

Step4: swap($arr[j], arr[j+1]$)

Step5: End

Program:

```
class Main{
    static void bubbleSort(int a[]){
        int len = a.length; // calculating the length of array
        for (int i = 0; i < len-1; i++)
            for (int j = 0; j < len-i-1; j++)
                if (a[j] > a[j+1]) //comparing the pair of elements
                {
                    // swapping a[j+1] and a[i]
                    int temp = a[j];
                    a[j] = a[j+1];
                    a[j+1] = temp;
                }
    }

    /* Prints the array */
    static void printArray(int a[]){
        int len = a.length;
        for (int i = 0; i < len; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }

    // Main method to test above
    public static void main(String args[]){
        int arr[] = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr); //calling the bubbleSort function
        System.out.println("Sorted array");
        printArray(arr); //calling the printArray function
    }
}
```

Output

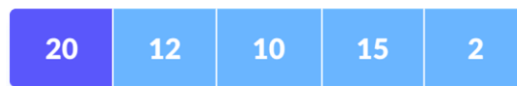
Sorted array

11 12 22 25 34 64 90

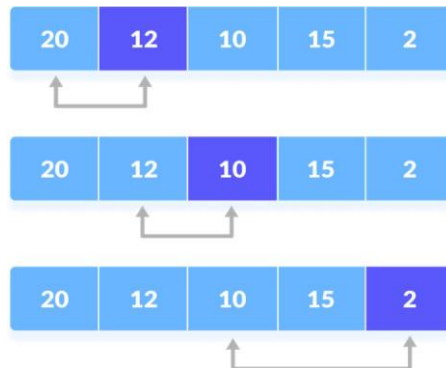
2. Selection Sort

Selection sort is a sorting algorithm that selects the smallest element from an unsorted list in each iteration and places that element at the beginning of the unsorted list.

Working of Selection Sort



1. Set the first element as minimum. Select first element as minimum
2. Compare minimum with the second element. If the second element is smaller than minimum, assign the second element as minimum.



Compare minimum with the third element. Again, if the third element is smaller, then assign minimum to the third element otherwise do nothing. The process goes on until the last element. Compare minimum with the remaining elements

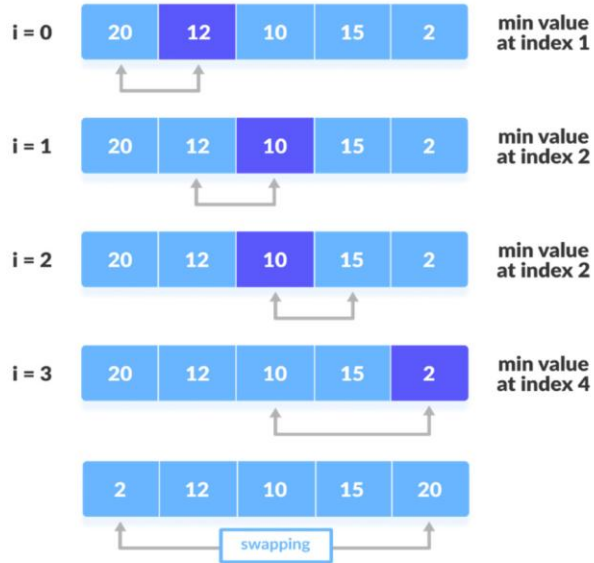
3. After each iteration, minimum is placed in the front of the unsorted list.



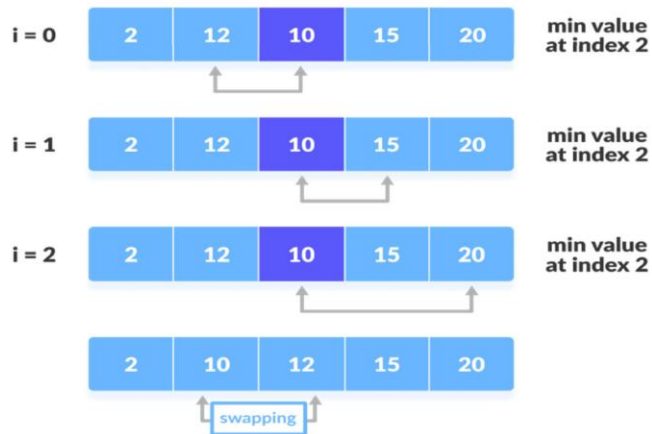
Swap the first with minimum

4. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.

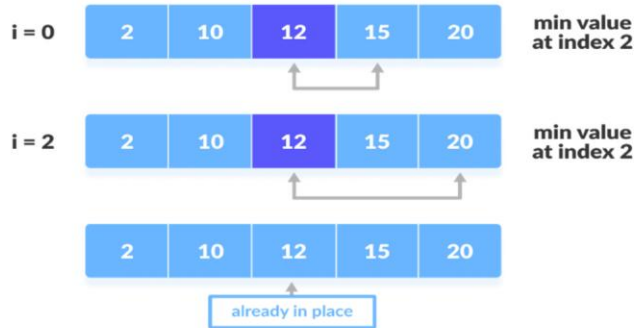
step = 0



step = 1



step = 2



step = 3



Selection Sort Algorithm

```
selectionSort(array, size)
  for i from 0 to size - 1 do
    set i as the index of the current minimum
    for j from i + 1 to size - 1 do
      if array[j] < array[current minimum]
        set j as the new current minimum index
    if current minimum is not i
      swap array[i] with array[current minimum]
  end selectionSort
```

Program:

```
// Selection sort in Java
import java.util.Arrays;
class SelectionSort {
  void selectionSort(int array[]) {
    int size = array.length;

    for (int step = 0; step < size - 1; step++) {
      int min_idx = step;
```

```
        for (int i = step + 1; i < size; i++) {

            // To sort in descending order, change > to < in this line.
            // Select the minimum element in each loop.
            if (array[i] < array[min_idx]) {
                min_idx = i;
            }

            // put min at the correct position
            int temp = array[step];
            array[step] = array[min_idx];
            array[min_idx] = temp;
        }

        // driver code
        public static void main(String args[]) {
            int[] data = { 20, 12, 10, 15, 2 };
            SelectionSort ss = new SelectionSort();
            ss.selectionSort(data);
            System.out.println("Sorted Array in Ascending Order: ");
            System.out.println(Arrays.toString(data));
        }
    }
```

Output:

```
Sorted Array in Ascending Order:
2
10
12
15
20
```

3. Insertion Sort:

Insertion Sort algorithm uses the technique of selecting a key in each iteration starting from the second element. Next, we make the proper position available for the key by shifting all the greater elements before it toward their right for sorting in ascending order. Finally, the selected key is stored before the last shifted greater element.

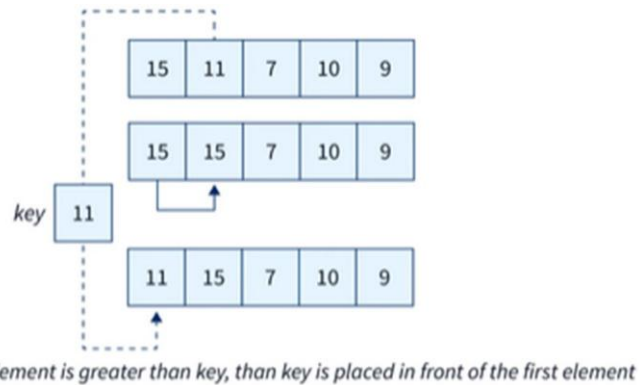
Let's understand this by sorting an array of integers, given as :

15	11	7	10	9
----	----	---	----	---

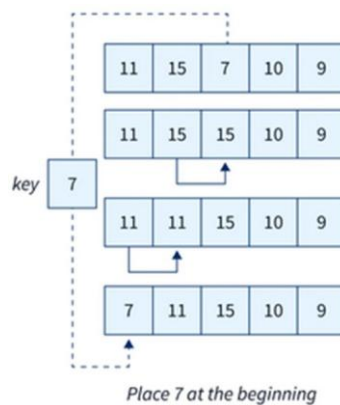
Initial Array

1. First Pass :

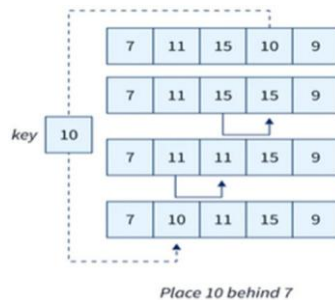
Assume the first element is sorted. Store the second element in the key variable. Compare the key with the first element, and place it before the first element if the first element is greater than the key. Do this by shifting elements before the key towards the right in the array.

**2. Second Pass :**

Now, store the third element in the key and compare it with the values on the left. Shift values more than the key to their right until you find a value smaller than or equal to the key. Store the key just before the last shifted element.

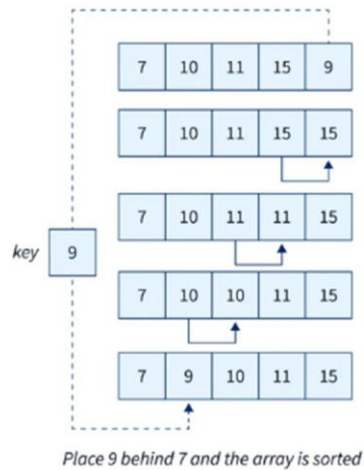
**3. Third Pass :**

Do similarly for the fourth element stored in the key.



4. Fourth Pass :

Finally, you get a sorted array for the fifth element as the key.

**Insertion Sort Algorithm**

1. Iterate for index 1 to n-1 of the list.
2. Now, compare the key (element at current index) with the preceding element.
3. If the key is smaller than the preceding element, compare it to other preceding elements. Shift greater elements one position to their right to make space for placing the key in the proper position.

Program:

```
import java.util.Arrays;

public class InsertionSort {

    public static void insertionSort(int[] arr) {
        int len = arr.length;

        for (int i = 1; i < len; i++) {
            // store the current value at current in key
            int key = arr[i];

            // pointer to index just before the key's index
            int j = i - 1;

            /* move values in arr[0...i-1] greater
             * than key one position to their right
             */
            while (j >= 0 && key < arr[j]) {
                arr[j + 1] = arr[j];
                j--;
            }

            /* store the key just before the last shifted element
             * or after the element just smaller than or equal to the key
             */
        }
    }
}
```

```
        arr[j + 1] = key;
    }
}

public static void main(String[] args) {
    int[] arr = { 15, 11, 7, 10, 9 };

    insertionSort(arr);

    System.out.println(Arrays.toString(arr));
}
}
```

Output :

[7, 9, 10, 11, 15]

Topic-8: Search for Values in Arrays

8.1 Linear Search

Linear search is a sequential searching algorithm where we start from one end and check every element of the list until the desired element is found. It is the simplest searching algorithm.

Algorithm

```
LinearSearch(array, key)
    for each item in the array
        if item == value
            return its index
```

// Linear Search in Java

Program:

```
class LinearSearch {
    public static int linearSearch(int array[], int x) {
        int n = array.length;

        // Going through array sequentially
        for (int i = 0; i < n; i++) {
            if (array[i] == x)
                return i;
        }
        return -1;
    }

    public static void main(String args[]) {
        int array[] = { 2, 4, 0, 1, 9 };
        int x = 1;

        int result = linearSearch(array, x);

        if (result == -1)
```

```
        System.out.print("Element not found");
    else
        System.out.print("Element found at index: " + result);
    }
}
```

Output:

Element found at index 3

8.2 Binary Search

Binary Search is a searching algorithm for finding an element's position in a sorted array.

In this approach, the element is always searched in the middle of a portion of an array.

Algorithm:

```
binarySearch(arr, x, low, high)
    if low > high
        return False
    else
        mid = (low + high) / 2
        if x == arr[mid]
            return mid
        else if x > arr[mid]           // x is on the right side
            return binarySearch(arr, x, mid + 1, high)
        else                          // x is on the left side
            return binarySearch(arr, x, low, mid - 1)
```

Program:

```
// Binary Search in Java

class BinarySearch {
    int binarySearch(int array[], int x, int low, int high) {

        if (high >= low) {
            int mid = low + (high - low) / 2;

            // If found at mid, then return it
            if (x == array[mid])
                return mid;

            // Search the right half
            if (x > array[mid])
                return binarySearch(array, x, mid + 1, high);

            // Search the left half
            return binarySearch(array, x, low, mid - 1);
        }

        return -1;
    }

    public static void main(String args[]) {
        BinarySearch ob = new BinarySearch();
        int array[] = { 3, 4, 5, 6, 7, 8, 9 };
        int n = array.length;
```

```
int x = 4;
int result = ob.binarySearch(array, x, 0, n - 1);
if (result == -1)
    System.out.println("Not found");
else
    System.out.println("Element found at index " + result);
}
}
```

Output:

Element found at index 1

Topic-9: Class Arrays

- The **Arrays** class in **java.util package** is a part of the **Java Collection Framework**.
- This class provides static methods to dynamically create and access **Java arrays**.
- It consists of only static methods and the methods of Object class.
- This class contains various methods for manipulating arrays (such as sorting and searching).
- The methods of this class can be used by the class name itself.

Arrays Class Declaration

Following is the declaration for **java.util.Arrays** class –

```
public class Arrays
    extends Object
```

Methods in Java Array Class

The static methods of the Java Array class could be used to perform operations like:

- Filling the elements
- Sorting the elements
- Searching for the elements
- Converting the array elements to String
- Many more..

Example that demonstrates the **binarySearch()** method of Arrays class:

```
// Importing the Arrays class
import java.util.Arrays;

// Public Main Class
public class Main {
    public static void main(String[] args) {
        // Declaring an array
        int arr[] = {8, 5, 3, 10, 2, 1, 15, 20};

        // Sorting the array
        Arrays.sort(arr);

        // Taking an element to search
        int ele = 15;
```

```

// Using binarySearch() method to search "ele"
System.out.println(
    ele + " presents at the index = " +
    Arrays.binarySearch(arr, ele));
}
}

```

Output:

15 presents at the index = 6

Topic-10: 2D Arrays in Java

2D array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns.

However, 2D arrays are created to implement a relational database look alike data structure. It provides ease of holding bulk of data at once which can be passed to any number of functions wherever required.

Declaring a 2D Array

The syntax of declaring two dimensional array is very much similar to that of a one dimensional array, given as follows.

```
int arr[max_rows][max_columns];
```

	0	1	2		n-1
0	a[0][0]	a[0][1]	a[0][2]		a[0][n-1]
1	a[1][0]	a[1][1]	a[1][2]		a[1][n-1]
2	a[2][0]	a[2][1]	a[2][2]		a[2][n-1]
3	a[3][0]	a[3][1]	a[3][2]		a[3][n-1]
4	a[4][0]	a[4][1]	a[4][2]		a[4][n-1]
.	.	.	.		.
.	.	.	.		.
n-1	a[n-1][0]	a[n-1][1]	a[n-1][2]		a[n-1][n-1]

a[n][n]

Initialize a 2d array

Here is how we can initialize a 2-dimensional array in Java.

```

int[][] a = {
    {1, 2, 3},
    {4, 5, 6, 9},
    {7},
};

```

	Column 1	Column 2	Column 3	Column 4
Row 1	1 a[0][0]	2 a[0][1]	3 a[0][2]	
Row 2	4 a[1][0]	5 a[1][1]	6 a[1][2]	9 a[1][3]
Row 3	7 a[2][0]			

Example: Print all elements of 2d array Using Loop

```
class MultidimensionalArray {
    public static void main(String[] args) {

        int[][] a = {
            {1, -2, 3},
            {-4, -5, 6, 9},
            {7},
        };

        for (int i = 0; i < a.length; ++i) {
            for(int j = 0; j < a[i].length; ++j) {
                System.out.println(a[i][j]);
            }
        }
    }
}
```

Output:

```
1
-2
3
-4
-5
6
9
7
```

We can also use the **for...each** loop to access elements of the 2-D array. For example,

```
class MultidimensionalArray {
    public static void main(String[] args) {

        // create a 2d array
        int[][] a = {
            {1, -2, 3},
            {-4, -5, 6, 9},
            {7},
        };

        // first for...each loop access the individual array
        // inside the 2d array
        for (int[] innerArray: a) {
```

```

        // second for...each loop access each element inside the row
        for(int data: innerArray) {
            System.out.println(data);
        }
    }
}

```

Output:

```

1
-2
3
-4
-5
6
9
7

```

Topic-11: 3-D Arrays in Java

Basically, a 3d array is an array of 2d arrays. The rows of a 3d array can also vary in length just like in a 2d array.

Initialize a 3d array in Java:

Let's see how we can use a 3d array in Java. We can initialize a 3d array similar to the 2d array. For example,

```

// test is a 3d array
int[][][] test = {
    {
        {1, -2, 3},
        {2, 3, 4}
    },
    {
        {-4, -5, 6, 9},
        {1},
        {2, 3}
    }
};

```

Example: 3-dimensional Array

```

class ThreeArray {
    public static void main(String[] args) {

        // create a 3d array
        int[][][] test = {
            {
                {1, -2, 3},
                {2, 3, 4}
            },
            {
                {-4, -5, 6, 9},
                {1},
            }
        };
    }
}

```

```

        {2, 3}
    }
};

// for..each loop to iterate through elements of 3d array
for (int[][] array2D: test) {
    for (int[] array1D: array2D) {
        for(int item: array1D) {
            System.out.println(item);
        }
    }
}
}
}

```

Output:

```

1
-2
3
2
3
4
-4
-5
6
9
1
2
3

```

Topic-12: Arrays as Vectors

- A vector is similar to a dynamic array whose size can be increased or decreased.
- Unlike arrays, it has no size limit and can store any number of elements.
- Since Java 1.2, it has been a part of the Java Collection framework. It's in the `java.util` package and implements the `List` interface

Creating a Vector

Here is how we can create vectors in Java.

```
Vector<Type> vector = new Vector<>();
```

Here, *Type* indicates the type of a linked list.

For example,

```

// create Integer type linked list
Vector<Integer> vector= new Vector<>();

// create String type linked list
Vector<String> vector= new Vector<>();

```

Methods of Vector

1. Add Elements to Vector

- `add(element)` - adds an element to vectors
- `add(index, element)` - adds an element to the specified position
- `addAll(vector)` - adds all elements of a vector to another vector

Example:

```
import java.util.Vector;

class Main {
    public static void main(String[] args) {
        Vector<String> mammals= new Vector<>();

        // Using the add() method
        mammals.add("Dog");
        mammals.add("Horse");

        // Using index number
        mammals.add(2, "Cat");
        System.out.println("Vector: " + mammals);

        // Using addAll()
        Vector<String> animals = new Vector<>();
        animals.add("Crocodile");

        animals.addAll(mammals);
        System.out.println("New Vector: " + animals);
    }
}
```

Output

```
Vector: [Dog, Horse, Cat]
New Vector: [Crocodile, Dog, Horse, Cat]
```

2. Remove Vector Elements

- `remove(index)` - removes an element from specified position
- `removeAll()` - removes all the elements
- `clear()` - removes all elements. It is more efficient than `removeAll()`

Example:

```
import java.util.Vector;

class Main {
    public static void main(String[] args) {
        Vector<String> animals= new Vector<>();
        animals.add("Dog");
        animals.add("Horse");
        animals.add("Cat");
```

```
        System.out.println("Initial Vector: " + animals);

        // Using remove()
        String element = animals.remove(1);
        System.out.println("Removed Element: " + element);
        System.out.println("New Vector: " + animals);

        // Using clear()
        animals.clear();
        System.out.println("Vector after clear(): " + animals);
    }
}
```

Output

```
Initial Vector: [Dog, Horse, Cat]
Removed Element: Horse
New Vector: [Dog, Cat]
Vector after clear(): []
```

3. Access Vector Elements

- `get(index)` - returns an element specified by the index
- `iterator()` - returns an [iterator](#) object to sequentially access vector elements

Example:

```
import java.util.Iterator;
import java.util.Vector;

class Main {
    public static void main(String[] args) {
        Vector<String> animals= new Vector<>();
        animals.add("Dog");
        animals.add("Horse");
        animals.add("Cat");
        // Using get()
        String element = animals.get(2);
        System.out.println("Element at index 2: " + element);
        // Using iterator()
        Iterator<String> iterate = animals.iterator();
        System.out.print("Vector: ");
        while(iterate.hasNext()) {
            System.out.print(iterate.next());
            System.out.print(", ");
        }
    }
}
```

Output

```
Element at index 2: Cat
Vector: Dog, Horse, Cat,
```